0. Préparer l'environnement

Installe Git depuis https://git-scm.com/downloads, puis ouvre Git Bash ou le terminal de VS Code.

Vérifie l'installation :

```
git --version
```

Configure ton identité (à faire une seule fois sur ton ordinateur) :

```
git config --global user.name "Ton Nom"
git config --global user.email "ton.email@example.com"
git config --global --list
```

Comprendre : Git inscrit le nom et l'adresse configurés dans les commits. Ce n'est pas la même chose que se connecter à GitHub.

Carte visuelle : le cycle Git

Chaque changement passe par trois zones. Le commit n'enregistre que les changements placés dans la zone de préparation.



À retenir : `git add` prépare. `git commit` enregistre localement. `git push` partage les commits sur le dépôt distant.

Comprendre le trajet des changements

Modifier un fichier

Ex. : ajouter un champ « prix » dans le formulaire produit.

↓ git diff

Examiner et préparer

Choisir exactement les fichiers à inclure.

```
git add ProductForm.jsx
```

↓ git diff --staged

Créer un point de sauvegarde

```
git commit -m "Ajouter le champ prix"
```

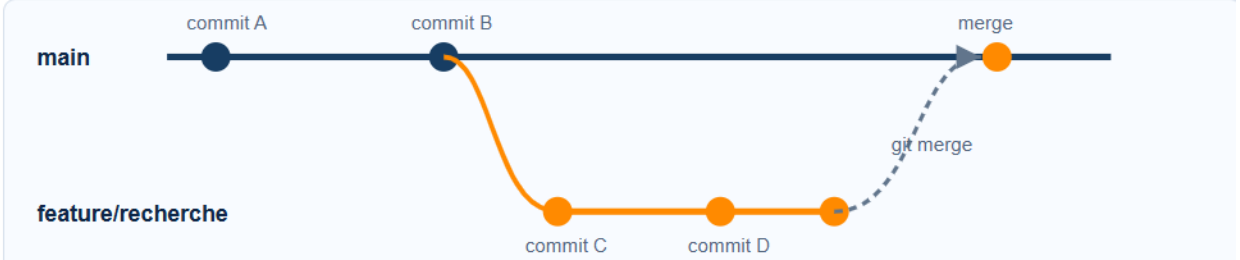
↓ si dépôt distant configuré

Partager avec l'équipe

```
git push
```

Branches : travailler sans casser la version stable

Une branche permet de développer une fonctionnalité séparément. Le merge intègre ensuite son historique dans la branche cible.



1. git switch -c feature/recherche
2. Modifier + commit · 3. Revenir sur main · 4. git merge feature/recherche

Choisir la bonne commande : arbre de décision

Que veux-tu faire ?

Voir l'état du projet `git status`	Examiner les modifications `git diff`	Préparer des changements `git add fichier`
Créer un point d'historique `git commit`	Changer de ligne de travail `git switch -c nom`	Partager les commits `git push`

PARTIE 1 — Comprendre le modèle mental de Git

1. Git, c'est quoi ?

Git est un système de contrôle de versions. Il permet de garder l'historique des changements, comparer des versions, créer des branches et collaborer.

Imagine que tu développes une application Laravel + React. Tu veux pouvoir répondre à ces questions :

- Qu'est-ce qui a changé depuis hier ?
- Quelle version fonctionnait avant le bug ?
- Puis-je développer une nouvelle fonctionnalité sans casser le code stable ?
- Comment intégrer le travail de mon collègue ?

Git aide à répondre à ces questions.

2. Les trois zones à comprendre absolument

Zone	Rôle	Image mentale
Working directory	Fichiers que tu modifies actuellement	Ton bureau de travail
Staging area / Index	Changements choisis pour le prochain commit	Le panier des changements à enregistrer
Repository local	Historique des commits enregistrés	Le journal des versions

Le cycle essentiel :

```
Modifier un fichier
       |
       v
Working directory
       |
       | git add
       v
Staging area
       |
       | git commit
       v
Repository local (historique)
```

Attention : `git add` ne crée pas un commit. Il prépare les changements. `git commit` enregistre un instantané des changements préparés dans l'historique local. `git push` envoie ensuite les commits vers un dépôt distant.

PARTIE 2 — Atelier 1 : créer ton premier dépôt

Situation

Tu commences un mini-projet de catalogue de produits. Tu veux que Git suive son évolution.

Étape 1 : créer le projet

Dans le terminal :

```
mkdir catalogue-produits
cd catalogue-produits
```

Crée un fichier `index.html` avec ce contenu :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Catalogue de produits</title>
</head>
<body>
  <h1>Mon catalogue</h1>
  <p>Bienvenue dans mon premier projet suivi avec Git.</p>
</body>
</html>
```

Étape 2 : initialiser Git

```
git init
git status
```

Comprendre :

- `git init` crée un dépôt Git local dans le dossier courant.
- `git status` indique la branche courante, les changements préparés et les fichiers non suivis.

Le fichier `index.html` apparaît normalement comme **untracked** : Git le voit, mais il n'est pas encore suivi dans un commit.

Étape 3 : préparer le fichier

```
git add index.html
git status
```

Observe le changement : le fichier est maintenant dans la staging area.

Pour préparer tous les changements du dossier courant :

```
git add .
```

N'utilise `git add .` qu'après avoir vérifié ce que tu prépares avec `git status`.

Étape 4 : créer le premier commit

```
git commit -m "Initialiser le catalogue"
git status
```

Un commit est un point de sauvegarde dans l'historique. Le message doit décrire le changement, par exemple : `Ajouter la page d'accueil`.

Étape 5 : inspecter l'historique

```
git log --oneline
git log
```

À retenir :

- `git log --oneline` affiche un historique compact.
- `git log` affiche plus de détails.
- Chaque commit possède un identifiant unique, souvent abrégé dans l'affichage.

Vérification de fin d'atelier

Tu dois savoir expliquer :

→ 1. Pourquoi `git init` ne sauvegarde-t-il pas les fichiers dans un commit ?
→ 2. Quelle différence entre `git add` et `git commit` ?
→ 3. Pourquoi utiliser `git status` avant un commit ?

PARTIE 3 — Atelier 2 : modifier et enregistrer proprement

Situation

Le client demande d'ajouter une description au catalogue.

Modifie `index.html` en ajoutant sous le paragraphe existant :

```
<p>Les produits sont classés par catégorie et par prix.</p>
```

Observe avant de préparer :

```
git status
git diff
```

Que montrent ces commandes ?

- `git status` résume les fichiers modifiés, non suivis ou préparés.
- `git diff` montre les changements non préparés par rapport au dernier état enregistré.
- `git diff --staged` montre les changements déjà préparés pour le prochain commit.

Prépare et enregistre :

```
git add index.html
git diff --staged
git commit -m "Ajouter une description au catalogue"
git log --oneline
```

Méthode professionnelle :

→ 1. Modifier.
→ 2. Examiner `git diff` .
→ 3. Préparer les changements.
→ 4. Examiner `git diff --staged` .
→ 5. Créer un commit avec un message clair.

Préparer seulement un fichier

Si tu as modifié `index.html` et `style.css` , mais que tu veux enregistrer seulement le HTML :

```
git add index.html
git status
git commit -m "Mettre à jour la page catalogue"
```

Le CSS reste modifié, mais non préparé. Un commit n'enregistre que ce qui est dans la staging area.

PARTIE 4 — Atelier 3 : comprendre les fichiers non suivis

Situation

Tu crées `notes.txt` avec quelques idées.

```
git status
git add notes.txt
git status
```

Avant `git add`, le fichier est non suivi. Après `git add`, il est préparé pour le prochain commit, mais il n'est pas encore enregistré dans l'historique.

Crée le commit :

```
git commit -m "Ajouter les notes du projet"
```

Le fichier `.gitignore`

Certains fichiers ne doivent généralement pas être versionnés : dépendances installées, caches, secrets ou fichiers générés.

Crée un fichier `.gitignore` à la racine du projet :

Le fichier `.gitignore`

Certains fichiers ne doivent généralement pas être versionnés : dépendances installées, caches, secrets ou fichiers générés.

Crée un fichier `.gitignore` à la racine du projet :

```
# Dépendances
node_modules/

# Variables d'environnement et secrets
.env
.env.*

# Fichiers générés
dist/
coverage/

# Éditeur et système
.vscode/
.DS_Store
Thumbs.db
```

Adapte ces règles au projet. Pour Laravel, le `.gitignore` standard du framework contient déjà des règles utiles. **Ne partage jamais de vrais mots de passe, tokens ou clés API.**

Puis :

```
git add .gitignore
git commit -m "Ignorer les fichiers générés et secrets"
```

Point important : `.gitignore` n'arrête pas le suivi d'un fichier déjà suivi par Git. Si un secret a déjà été commité, l'ajout dans `.gitignore` ne l'efface pas de l'historique ; il faut révoquer le secret et traiter l'historique séparément.

PARTIE 5 — Atelier 4 : annuler sans paniquer

Avant toute commande qui peut supprimer ou remplacer des changements, commence par :

```
git status
git diff
```

Cas A : annuler une modification non préparée

Tu modifies `index.html`, puis tu décides de jeter ces modifications locales :

```
git restore index.html
```

Cette commande remplace les modifications non préparées par la version de l'index. Les changements non enregistrés dans ce fichier seront perdus.

Ne l'exécute pas si tu veux conserver ton travail.

Cas B : retirer un fichier de la staging area

Tu as préparé un fichier par erreur :

```
git restore --staged index.html
```

Le fichier sort de la staging area, mais ses modifications restent dans le working directory.

Cas C : corriger le dernier message de commit

Si tu viens de créer un commit et que tu veux corriger son message :

```
git commit --amend -m "Message corrigé"
```

À utiliser avec prudence si le commit a déjà été partagé, car cette opération réécrit le commit.

Cas D : annuler un commit déjà partagé

Pour annuler les effets d'un commit tout en ajoutant un nouveau commit d'annulation :

```
git revert <identifiant_du_commit>
```

Remplace le texte entre chevrons par l'identifiant réel. Cette approche est souvent préférable pour un historique partagé.

Différence importante :

- `git restore` : agit sur les fichiers.
- `git revert` : crée un commit qui inverse les changements d'un commit précédent.
- `git reset` : déplace la référence de branche et peut rendre des commits inaccessibles depuis cette branche ; à étudier séparément, avec précaution.

PARTIE 6 — Atelier 5 : travailler avec les branches

Pourquoi une branche ?

Tu veux ajouter une fonctionnalité sans modifier directement la branche stable. Une branche est une ligne de développement indépendante.

Vérifie ta branche :

```
git branch
```

Crée et sélectionne une branche :

```
git switch -c feature/recherche-produits
```

Ajoute une fonctionnalité dans le projet, puis :

```
git status
git add .
git commit -m "Ajouter la recherche de produits"
```

Reviens à la branche principale :

```
git switch main
```

Si ta branche principale s'appelle `master`, utilise `master`. Le nom réel se vérifie avec `git branch`.

Intègre la fonctionnalité :

```
git merge feature/recherche-produits
```

Après vérification, tu peux supprimer la branche locale fusionnée :

```
git branch -d feature/recherche-produits
```

Comprendre : `git switch` change la branche courante. `git merge` intègre les changements d'une autre branche dans la branche courante.

PARTIE 7 — Atelier 6 : résoudre un conflit de fusion

Situation

Deux développeurs modifient la même ligne d'un fichier sur deux branches différentes. Git ne peut pas décider automatiquement quelle version conserver.

➜ 1. Crée deux branches à partir d'un état commun.
➜ 2. Modifie la même ligne du même fichier dans chacune.
➜ 3. Crée un commit sur chaque branche.
➜ 4. Fusionne la deuxième branche dans la première.

Git peut afficher des marqueurs comme :

```
<<<<<<< HEAD
Version de la branche courante
=======
Version de l'autre branche
>>>>>>> autre-branche
```

Méthode de résolution

➜ 1. Ouvre le fichier en conflit.
➜ 2. Choisis ou combine les versions utiles.
➜ 3. Supprime les marqueurs de conflit.
➜ 4. Vérifie le fichier final.
➜ 5. Prépare le fichier résolu et termine la fusion.

```
git status
git add nom-du-fichier
git commit
```

Selon le type de fusion, Git peut ouvrir un éditeur pour le message de commit. Vérifie toujours `git status`.

Pour abandonner une fusion en cours si Git le permet :

```
git merge --abort
```

Cette commande tente de revenir à l'état antérieur au début de la fusion. Vérifie d'abord que ton travail local est sauvegardé.

PARTIE 8 — Atelier 7 : GitHub, dépôt distant et collaboration

Git est le système de versions ; GitHub est une plateforme qui héberge des dépôts Git et facilite la collaboration.

Lier un dépôt local à un dépôt distant

Crée d'abord un dépôt vide sur GitHub. Puis remplace l'URL d'exemple par l'URL de ton dépôt :

```
git remote add origin https://github.com/UTILISATEUR/catalogue-produits.git
git remote -v
```

Si ta branche principale est `main` :

```
git branch -M main
git push -u origin main
```

Comprendre :

- `git remote -v` affiche les dépôts distants configurés.
- `git push` envoie les commits locaux vers le dépôt distant.
- `-u` configure la branche distante de suivi pour simplifier les prochains push.

Récupérer le travail partagé

```
git pull
```

`git pull` récupère les changements distants et les intègre à la branche courante. Avant de l'utiliser, vérifie ta branche et ton état local avec `git status`.

Pour un nouveau membre qui veut récupérer le projet :

```
git clone https://github.com/UTILISATEUR/catalogue-produits.git
```

`git clone` crée une copie locale d'un dépôt distant avec son historique.

Workflow d'équipe conseillé

```
git switch main
git pull
git switch -c feature/nom-fonctionnalite
# Modifier et vérifier les fichiers
git status
git diff
git add fichier1 fichier2
git diff --staged
git commit -m "Décrire le changement"
git push -u origin feature/nom-fonctionnalite
```

Ensuite, ouvre une Pull Request sur GitHub, demande une revue de code et fusionne selon les règles de l'équipe.

PARTIE 9 — Les commandes à connaître par cœur

Commande	Question à laquelle elle répond
`git init`	Comment commencer un dépôt local ?
`git clone URL`	Comment récupérer un dépôt distant ?
`git status`	Où en suis-je dans le cycle Git ?
`git add fichier`	Quels changements préparer ?
`git commit -m "message"`	Quel instantané enregistrer ?
`git diff`	Qu'est-ce qui a changé sans être préparé ?
`git diff --staged`	Qu'est-ce qui sera enregistré au prochain commit ?
`git log --oneline`	Quels commits ont été enregistrés ?
`git switch -c nom`	Comment créer et ouvrir une branche ?
`git switch main`	Comment changer de branche ?
`git merge branche`	Comment intégrer une branche ?
`git pull`	Comment récupérer et intégrer les changements distants ?
`git push`	Comment envoyer mes commits ?
`git restore fichier`	Comment annuler des modifications non préparées ?
`git restore --staged fichier`	Comment retirer un fichier de la staging area ?
`git revert ID`	Comment annuler un commit via un nouveau commit ?

PARTIE 10 — Évaluation pratique

Exercice 1 — Individuel

Sans copier les commandes précédentes :

➜ 1. Crée un dossier `gestion-stagiaires` .

➜ 2. Initialise Git.

➜ 3. Ajoute un fichier `README.md` présentant le projet.

➜ 4. Crée un premier commit.

➜ 5. Modifie le README.

➜ 6. Utilise `git diff` pour examiner la modification.

➜ 7. Prépare et committe la modification.

➜ 8. Affiche l'historique compact.

Livrable attendu : le dossier du projet et les deux commits visibles avec `git log --oneline` .

Exercice 2 — Branches

➜ 1. Crée une branche `feature/ajout-stagiaire`.

➜ 2. Ajoute un fichier décrivant un stagiaire fictif sans données personnelles réelles.

➜ 3. Crée un commit.

➜ 4. Reviens sur `main`.

➜ 5. Fusionne la branche.

➜ 6. Vérifie le résultat et l'historique.

Exercice 3 — Diagnostic

Pour chaque situation, indique la commande ou la méthode adaptée :

- Tu veux savoir quels fichiers ont été modifiés.
- Tu veux voir la différence exacte avant de préparer.
- Tu as préparé le mauvais fichier, mais tu veux garder ses modifications.
- Tu veux isoler une fonctionnalité dans une branche.
- Tu veux envoyer tes commits sur GitHub.
- Tu as un conflit de fusion et tu dois garder une version corrigée des deux développeurs.

Barème suggéré sur 20 points

- Initialisation et configuration : 3 points
- Cycle `status → diff → add → commit` : 5 points
- Historique et messages de commit : 3 points
- Branches et fusion : 4 points
- Diagnostic et explication des choix : 5 points